finora.

WEB APPLICATION DEVELOPMENT · MINI PROJECT

Personal Expense Tracker

A simple, secure workspace for recording personal income and expenses and understanding monthly cash flow.

PROJECT
Finora — Personal Expense Tracker

REPORT DATE
22 October 2026

APPLICATION TYPE
Full-stack web application

SUBMISSION
University mini-project report

Introduction

Finora is a browser-based personal finance tracker. It brings everyday income and expense records, user-defined categories, and period summaries into one focused interface.

Problem statement

Personal spending recorded across paper notes or disconnected spreadsheets can be difficult to review consistently. Users need a straightforward way to maintain a transaction history, group expenses, and see how recorded income compares with spending.

Project objectives

1. Provide authenticated accounts and keep each user's categories and transactions scoped to that account.
2. Support creating, viewing, editing, and deleting income and expense records with input validation.
3. Make transactions easier to review using search, date and category filters, sorting, and pagination.
4. Summarize recorded activity in a dashboard and date-based reports using backend data.
5. Keep the application understandable, responsive, and practical to run locally for coursework.

> **Scope:** Finora tracks user-entered financial records. Currency selection changes formatting only; it does not perform exchange-rate conversion or connect to a bank.

Implemented feature set

Accounts
Register, log in, edit profile details, and sign out using JWT authentication.

Transactions
Manage income and expenses with categories, dates, descriptions, filters, sorting, and pagination.

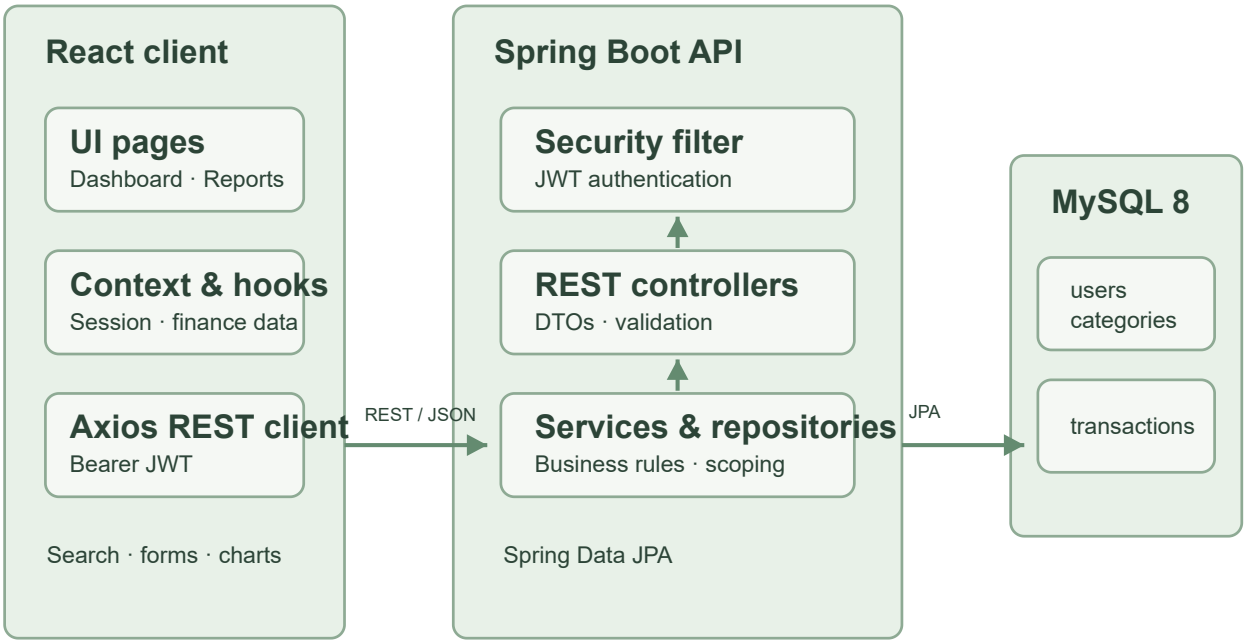
Insights
Review lifetime totals, monthly cash flow, recent transactions, category spending, and reports.

Technology stack

Layer	Technologies	Purpose
Frontend	React 18, Vite, Axios, CSS	Responsive single-page application and REST client.
Backend	Java 21, Spring Boot, Spring Security, Spring Data JPA	REST API, validation, authentication, and business rules.
Persistence	MySQL 8, Flyway	Relational storage and schema migration.
Verification	Vitest, React Testing Library, JUnit 5, H2, ESLint	Frontend tests, backend tests, and static checks.

System architecture

The application uses a conventional client–server design. The React client calls the Spring Boot REST API; backend services apply business rules and user scoping before repositories access MySQL.



Request flow

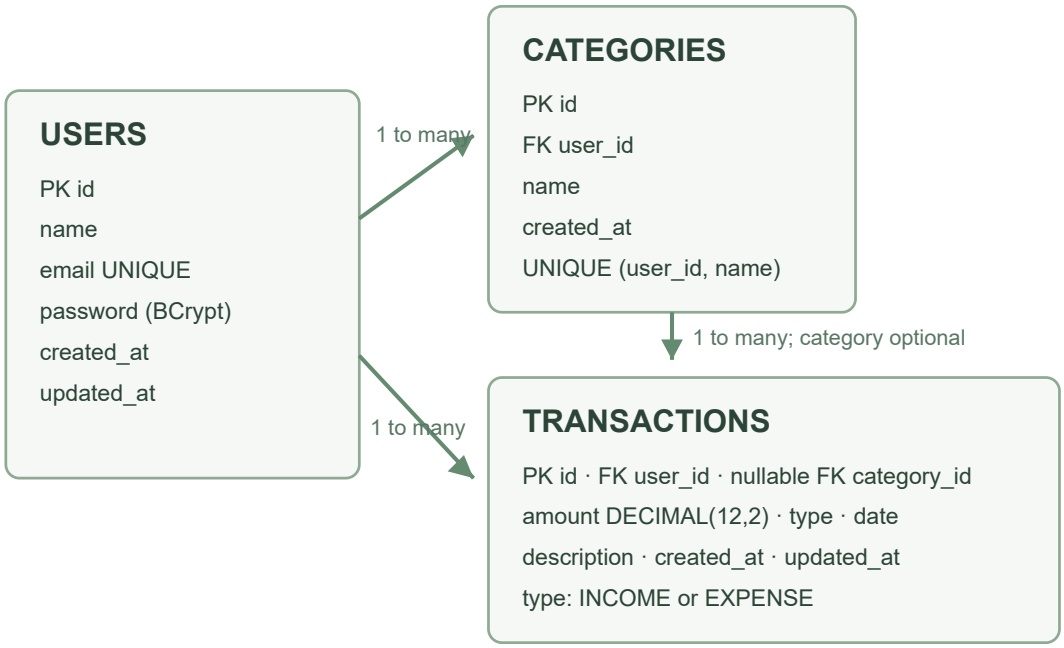
A signed-in browser sends a REST request with a bearer token. Spring Security validates the token; controllers validate request DTOs and services apply the authenticated user's scope.

Data access

Spring Data repositories persist users, categories, and transactions in MySQL. Monetary values use decimal database types and Java decimal values rather than binary floating-point storage.

Database ERD

The relational model has three core entities. User ownership is explicit for categories and transactions; a transaction may have an optional category reference.



Ownership and integrity

A user owns many categories and transactions. Per-user category names are unique. User deletion cascades to owned records; a category referenced by transactions cannot be deleted.

Financial records

Transactions belong to one user. Expenses require a category in the application; income may be uncategorized. Amounts are positive and stored to two decimal places.

Schema source: Flyway migration `V1__init_schema.sql`. Actual column names are represented in snake_case.

Working application screenshot

The following image was captured from the application running locally in a browser. It shows the actual Finora authentication screen, not a mock-up.

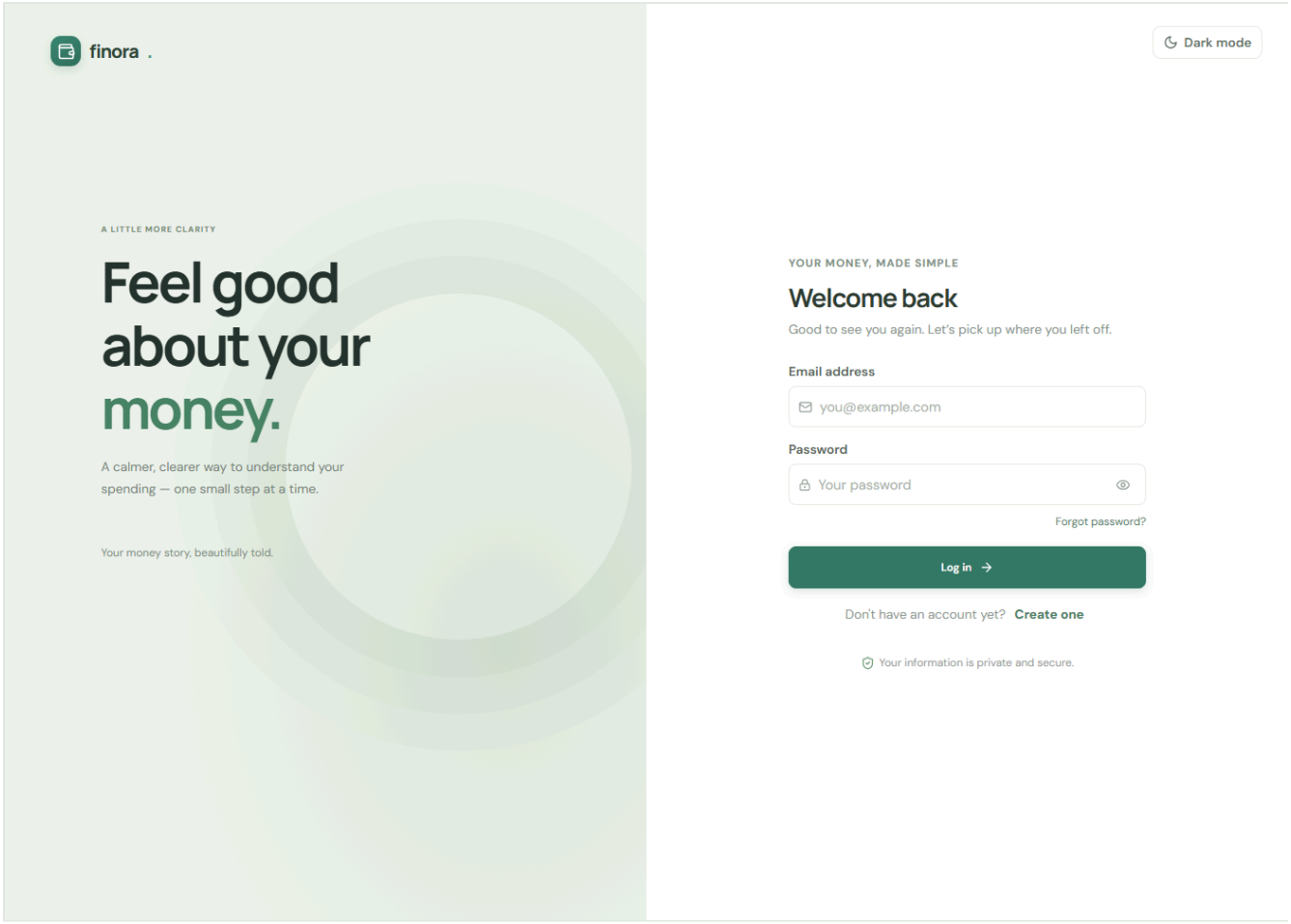


Figure 1. Finora sign-in screen, captured from the locally running Vite application on 22 October 2026.

> **Evidence limitation:** The authenticated dashboard and finance pages require a running backend and a real account with data. Those were not available for this capture, so the next page identifies the exact screenshots to add before final submission.

Authenticated-page screenshots

Replace each clearly marked placeholder with a screenshot captured from your own running application and account. Keep values authentic; do not use illustrative or fabricated financial records.

PLACEHOLDER · Dashboard

Capture the signed-in Overview page with live totals, current-month cash flow, category breakdown, and recent transactions.

PLACEHOLDER · Transactions

Capture the transaction list with real records and visible search/filter/sort controls. Avoid exposing private account details.

PLACEHOLDER · Categories

Capture the category page with actual categories and their displayed monthly spend or empty state.

PLACEHOLDER · Reports

Capture a report for a visible month or chosen date range, including actual summary values and category totals.

Capture steps: Start MySQL, the backend, and the frontend using the root README. Sign in with your own account, navigate to each named page, and capture the browser window. Replace the matching placeholder block in this HTML source with each screenshot, then print/export this file to PDF again.

05 · VERIFICATION

Testing approach and results

Automated checks were run locally on 22 October 2026. Backend tests use H2; they do not require access to or alter the MySQL development database.

Check	Verified result	Coverage / scope
Frontend tests `npm --prefix Front-End run test`	**31 passed · 6 files**	Dashboard calculations, transaction-page rendering, validation, date/currency formatting, and CSV export.
Frontend lint `npm --prefix Front-End run lint`	**Passed**	ESLint checks the frontend source.
Frontend build `npm --prefix Front-End run build`	**Passed**	Vite produced the production bundle.
Backend tests `.\Back-End\expense-tracker\mvnw.cmd -f .\Back-End\expense-tracker\pom.xml test`	**29 passed · 0 failures**	Authentication, service rules, reports, transaction sorting/pagination, and user isolation using JUnit 5 and H2.

Verification environment: Node.js 24.13.0 and Java 25 were installed locally; the project targets Java 21. Re-run the checks using the required Java 21 setup before submission.

Manual verification still required

The automated suites do not replace a browser-based end-to-end acceptance run. Before demonstration, sign in with a local account and manually verify registration/login, transaction and category CRUD, delete confirmations, filters/sorting/pagination, report date ranges, theme switching, currency formatting, and desktop/mobile layouts.

> **Verification boundary:** No authenticated browser flow against a live MySQL-backed API or visual comparison at multiple screen sizes was run for this report. The test results above describe only the commands and test scopes that were executed.

Limitations and conclusion

Known limitations

- Currency choices are presentation formats only; exchange conversion and live exchange rates are not implemented.
- Password recovery/change, recurring transactions, budget management, bank integrations, and outbound notifications are not implemented. The monthly summary preference is stored locally only.
- Reports depend on the user's recorded data and supported date filters; the product is not financial or tax advice.
- The authenticated product screenshots remain placeholders until captured using a live local account.
- Docker configuration is optional and was not validated as part of this report.

Conclusion

Finora demonstrates a complete, approachable personal-expense workflow across a React frontend, a Spring Boot REST backend, and a relational MySQL data model. It combines user-scoped records, category management, summary views, and automated tests while keeping the local development setup straightforward.

The verified test and build results provide evidence for core component, service, API security, filtering, and calculation behavior. Final academic submission should include the student's own authenticated-page screenshots and a manual browser walkthrough using the supplied local setup instructions.

Local run summary

```
1. Start MySQL 8 and create the expense_tracker database.
2. Set DB_URL, DB_USERNAME, DB_PASSWORD, and a private JWT_SECRET.
3. Run .\Back-End\expense-tracker\mvnw.cmd spring-boot:run
4. In Front-End, run npm ci and npm run dev.
5. Open the Vite URL shown in the terminal (usually http://localhost:5173).
```

Docker is not required. See the repository README for prerequisites, environment configuration, complete test commands, and backend-specific notes.